

Case 1: an FCNN surrogate for the SPE11B pairwise distance

1 Problem

The SPE11B benchmark compares submissions through a pairwise distance table: for every reporting year, the Wasserstein-1 distance between the CO₂ mass images of each two submissions. We train a network

$$f_{\theta} : \mathbb{R}^{2 \times 120 \times 840} \rightarrow \mathbb{R}, \quad f_{\theta}(M, M') \approx d_t(M, M'), \quad (1)$$

to reproduce that table from the two images alone, as a cheap surrogate for evaluating d_t .

2 Notation

$\mathcal{G} = \{g_0 < \dots < g_{33}\}$	the $G = 34$ submissions, alphabetically
$\mathcal{T} = \{0, 5, \dots, 1000\}$	the $T = 201$ reporting years
$M \in \mathbb{R}^{120 \times 840}$	one image: a field of CO ₂ mass, in kg per cell
$D^{(t)} \in \mathbb{R}^{34 \times 34}$	ground-truth distance table at year t

$D^{(t)}$ is symmetric with zero diagonal, and exists for $t \neq 0$ only: 200 of the 201 years.¹

3 Data

Images. A single archive² holds one image per column, flattened row-major over the 840×120 grid, its column index k being `index_image_in_archive` in the code:

$$A \in \mathbb{R}^{100800 \times 6833}, \quad M_k := A_{:,k} \text{ reshaped}, \quad (M_k)_{r,c} = A_{840r+c, k}. \quad (2)$$

Rows run bottom-up ($r = 0$ is the deepest layer), so plotting requires `origin="lower"`.

Column labels. The companion file `spe11b_tmco2_dt50y_indices.json` is a list of labels, not a numeric array:

$$\mu : \{0, \dots, 6832\} \rightarrow \mathcal{G} \times \mathcal{T}, \quad \mu(k) = (g_k, \tau_k). \quad (3)$$

It is a bijection onto $(\mathcal{G} \times \mathcal{T}) \setminus \{(\text{kfupm1}, 815)\}$, the one image missing from the source data; hence the column count

$$6833 = GT - 1 = 34 \cdot 201 - 1. \quad (4)$$

¹One table per file `dense/spe11b_co2mass_w1_diff_ty.csv`. Their row and column labels drop the trailing 1 of single-submission groups (`calgary` for `calgary1`); `_distance_lookup` undoes the relabelling.

²`spe11b_tmco2_dt50y_images.npz`, built by `process_map_files.py`. The suffix `dt50y` is misleading: the interval in the file is 5 years.

Ordering. Columns are sorted by year, then alphabetically by group. With $\delta_k := \mathbf{1}[k \geq 5551]$ correcting for the missing image,

$$\tau_k = 5 \lfloor (k + \delta_k) / G \rfloor, \quad g_k = g_{(k + \delta_k) \bmod G}. \quad (5)$$

The year blocks $\mathcal{I}_t := \{k : \tau_k = t\}$ are therefore contiguous, of size 34 (33 for $t = 815$):

$$\mathcal{I}_{5s} = \{34s, \dots, 34s + 33\}, \quad s \leq 162. \quad (6)$$

4 Dataset

The four steps below build the training pairs. Their symbols carry the following names in `utils_datasets.py`:

k	<code>index_image_in_archive</code>
$\pi(k)$	<code>index_image</code>
ℓ , a row of $\mathcal{P}$	<code>index_pair</code>
$\mathcal{P}$	<code>index_image_pairs</code>

1. Column selection. `create_datasets` keeps $\mathcal{K} := \text{range}(\text{start}, \text{stop}, \text{step})$, $n := |\mathcal{K}|$, and re-indexes it by position,

$$\pi(k) = \frac{k - \text{start}}{\text{step}} \in \{0, \dots, n - 1\}, \quad X \in \mathbb{R}^{n \times 120 \times 840}, \quad X_{\pi(k)} = M_k. \quad (7)$$

In `main.py`, `start = 34 = G` drops exactly the year-0 block $\mathcal{I}_0$, which has no $D^{(t)}$, and `step = 1`, so $\pi(k) = k - 34$ and $n = 6799$.

2. Pairing, within a year. With $\mathcal{K}_t := \mathcal{K} \cap \mathcal{I}_t$ and $n_t := |\mathcal{K}_t| \leq G$,

$$\mathcal{P} := \biguplus_t \pi(\mathcal{K}_t) \times \pi(\mathcal{K}_t), \quad N := |\mathcal{P}| = \sum_t n_t^2. \quad (8)$$

Both orderings and the diagonal are included; no pair crosses years.

Why within a year: a constraint of the data, not a modelling choice. The benchmark publishes one 34×34 table per year, so $D_{pq}^{(t)}$ is defined only for two submissions compared *at the same t* . No ground-truth distance exists between images at different times, e.g. between (`calgary1,5y`) and (`sintef1,900y`). Pairing across timesteps would require computing the Wasserstein-1 distance ourselves, which is done nowhere in this repository.

3. Targets. For $\ell = (\pi(i), \pi(j)) \in \mathcal{P}$,

$$y_\ell := D_{p(i), p(j)}^{(\tau_i)}, \quad p(k) := (k + \delta_k) \bmod G \text{ the rank of } g_k \text{ in } \mathcal{G}. \quad (9)$$

4. Scaling and splitting. Inputs and targets are min-max scaled to $[0, 1]$ independently, with extrema taken over the whole dataset; the rows of $\mathcal{P}$ are then cut into contiguous 70/15/15 slices, unshuffled.

Index bounds. A row of $\mathcal{P}$ has both entries in $\{0, \dots, n - 1\}$, but within one year block they range only over the n_t consecutive positions of that block. For `main.py` the block of year $5s$ occupies rows $1156(s - 1), \dots, 1156s - 1$ with entries in $\{34(s - 1), \dots, 34(s - 1) + 33\}$, so the array opens with

$$(0, 0), (0, 1), \dots, (0, 33), (1, 0), \dots, (33, 33), (34, 34), \dots \quad (10)$$

The bound 33 is $G - 1$: the submissions available at one year, not a number of timesteps. A single submission contributes $T = 201$ images, but spread over 201 different blocks.

Counts. For `main.py`, $n_t = 34$ for all years but $n_{815} = 33$, so

$$N = 199 \cdot 34^2 + 33^2 = 231133 \longrightarrow 161793 / 34669 / 34671. \quad (11)$$

Two consequences of the choices above are worth stating. Scaling before splitting makes the two extrema statistics of the test set as well. And since the rows of $\mathcal{P}$ are ordered by year, a contiguous split is a *temporal* one: training covers years 5–700, validation 700–855, test 855–1000, so the model is assessed by extrapolation in time.

The three sets share one image stack X ; only $\mathcal{P}$ is split, so memory grows with n and not with N .

5 Model

A pair is flattened to $x_\ell \in \mathbb{R}^d$, $d = 2 \cdot 120 \cdot 840 = 201600$, and passed through a fully connected network of widths $[d, 64, 64, 1]$. Activations are SiLU, $\text{silu}(x) = x \sigma(x)$, on every layer but the last, which is linear. Weights use He initialization, $W \sim \mathcal{N}(0, 2/\text{fan_in})$; biases start at zero.

6 Training

Adam on the mean squared error over minibatches B ,

$$\mathcal{L}(\theta) = \frac{1}{|B|} \sum_{\ell \in B} (f_\theta(x_\ell) - y_\ell)^2, \quad (12)$$

with the training rows reshuffled each epoch. `main.py` uses $|B| = 128$, learning rate 10^{-2} and 100 epochs, i.e. 1264 steps per epoch; losses are logged every 10.

7 Test metrics

Let $e_i := \hat{y}_i - y_i$ be the residual on the i -th of the N test pairs and $\bar{y}$ the mean target. `postprocess.py` reports

$$\text{NMSE} = \frac{\sum_i e_i^2}{\sum_i y_i^2}, \quad \text{NMAE} = \frac{\sum_i |e_i|}{\sum_i y_i}, \quad (13)$$

$$\text{NRMSE} = \sqrt{\text{NMSE}}, \quad R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2}. \quad (14)$$

The first three are each normalized by a statistic of the targets, hence dimensionless; they are *not* the plain MSE, RMSE and MAE, despite the names `mse`, `rmse`, `mae` carrying them in the code. When $\sum_i (y_i - \bar{y})^2 = 0$, the convention is $R^2 = 1$ if the residuals vanish and $R^2 = 0$ otherwise. `plot_r_squared` scatters $\hat{y}_i$ against y_i with the identity line and annotates R^2 .